Contents

Project Documentation — Lumen DS Cross-Sell & Up-Sell Recommendation System **1**
Table of Contents . . . 1
1. Business Understanding . . . 1
2. Data Understanding . . . 2
3. Data Preparation . . . 5
4. Modeling . . . 7
5. Evaluation . . . 11
6. Deployment . . . 13
Appendix — Pipeline Output Summary . . . 15

Project Documentation — Lumen DS Cross-Sell & Up-Sell Recommendation System

Methodology: CRISP-DM (Cross-Industry Standard Process for Data Mining)

This document describes the end-to-end data-science process behind the Lumen DS competition project — from understanding the business problem through deploying a real-time recommendation API. It complements the operational README.md and RUN_BOOK.md, which focus on *how to run* the pipeline; here we document *what we built and why*.

Table of Contents

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Deployment

---

1. Business Understanding

1.1 Why Cross-Sell and Up-Sell Matter

Customer-base monetization is one of the highest-leverage activities in B2B distribution. Industry research consistently shows that:

- **Acquiring a new customer costs 5–25× more than retaining an existing one.** Existing customers already trust the brand, are integrated into procurement workflows, and have an account in the ERP — none of which a cold lead has.
- **The probability of selling to an existing customer is 60–70%, compared with 5–20% for a new prospect.** Even a modest improvement in cross-sell hit rate compounds into meaningful revenue uplift.
- **Up-selling existing customers to higher-tier items raises average order value without raising acquisition spend.** It directly improves gross margin per customer relationship.

For a B2B distributor with ~12,000 active accounts, the question is not *whether* to invest in cross-sell and up-sell, but *which customer should be offered which product, and when.*

1.2 Project Goal

Build a data-driven recommendation system that:

1. **Segments customers** into behavioral groups (buyer personas) so that recommendations can be personalized to each segment.

2. **Discovers product associations** in the historical transaction data to identify which products are commonly bought together.
3. **Serves recommendations in real time** through a REST API that the sales team or e-commerce front-end can call when a customer is on the phone or at checkout. Two endpoints are required:
   - `POST\/cross-sell` — given a customer (and optionally items currently being ordered), recommend additional product groups they are likely to buy.
   - `POST\/up-sell` — given specific items the customer is buying, recommend higher-tier alternatives in the same product group.

1.3 Success Criteria

Dimension	Target	Why
Segmentation quality	Silhouette score interpretable, segments behaviorally distinct	Sales team must be able to act on the segments
Cross-sell ranking	ROC-AUC ≥ 0.85 with no temporal leakage	Ranking quality matters more than absolute precision in a 290:1 imbalance setting
Up-sell signal	Positive price-uplift evidence in a held-out window	We cannot run an A/B test, so a forward-time proxy is the best available evidence
API latency	Sub-second per call, all artifacts in memory	The API is hit during live sales conversations
Reproducibility	Fixed seeds, documented temporal split	A re-run must produce identical artifacts

1.4 Constraints and Assumptions

- **Data is the only source of truth.** No domain expert is available to label "this is a cross-sell pair" — associations must be mined from transaction history.
- **The dataset is B2B.** This is established empirically in §2.4 and shapes every modeling choice (customer-level basket, segment-conditional rankings, focus on key accounts).
- **No A/B test infrastructure is available.** Validation must rely on temporal hold-out (train on the past, evaluate on the future) rather than experimental treatment vs. control.
- **The API must be simple.** Per the project brief: two endpoints, accepting `CustomerID` and a list of products, returning ranked recommendations with confidence scores.

---

2. Data Understanding

2.1 Source Dataset

Property	Value
Filename	`LUMEN_DS.csv`
Encoding	UTF-16, pipe-separated, ~795 MB
Rows	~1,300,000 invoice line items
Columns	32 raw fields covering customer, product, transaction, financial, sales-channel, and manufacturing dimensions
Granularity	One row per invoice line (one product on one invoice)

2.2 Columns Used in the Pipeline

The raw dataset has 32 columns, but the modeling pipeline only consumes a focused subset. The table below lists every column that is actually read by at least one notebook, together with the role it plays. Columns not listed are not used by any model — they are summarized in §2.3.

"Where used" refers to the production pipeline: `01\data_cleaning`, `02\customer_segmentation`, `03\market_basket`, `04\cross_sell_recommender`, `05\upsell_model`. The two notebooks under `notebooks/archive/` are exploratory and not part of the API path — see §3.7 and §4.3.1.

Column	Where Used	Role in This Project
`CustomerID`	All notebooks	Primary key for every per-customer analysis (RFM, segmentation, basket grouping, recommendations).
`Invoice\Date`	All notebooks	Recency in RFM (segmentation), customer-month basket key (market basket), train/label split cutoff (cross-sell, up-sell), tenure calculation (cross-sell).
`Invoice\#`	Segmentation, market basket, cross-sell	Frequency in RFM; basket co-occurrence unit at the invoice level (used to confirm invoice-level baskets are too sparse and to switch to customer-month); `train_orders` feature for the cross-sell model.
`Invoiced\price`	Cleaning, segmentation, cross-sell, up-sell	Monetary in RFM (with quantity); imputed where ≥ 50% of an item's rows have valid prices; item average price for tier classification (up-sell); `pg_avg_price` feature (cross-sell).
`Invoiced\qty\(shipped)`	Cleaning, segmentation, cross-sell	Multiplied with `Invoiced\price` to compute `line_revenue` (Monetary, `train_revenue`, `pg_avg_revenue`). Negative or zero rows are filtered in cleaning.
`Item\Code`	Cleaning, market basket (bonus item-level run), cross-sell, up-sell	Product identifier; `train_items` feature; tier classification target; Layer-2 item recommendation level inside the API. `SAMPLE` and `/`-prefixed codes are filtered in cleaning.
`Product\group`	Market basket, cross-sell, up-sell	Recommendation granularity for the cross-sell ranker; basket-level co-occurrence unit; PG-level price median for up-sell tiering.
`GM%`	Cleaning, cross-sell	Recomputed from price and cost where both are available; winsorized to `{[}-5,\1{]}`; `train_avg_gm` and `pg_avg_gm` features.
`Customer\Region`	Cross-sell	Label-encoded → `region_enc` feature.
`Customer\industry`	Cross-sell	Label-encoded → `industry_enc` feature.
`Top\Customer\Group`	Cross-sell	Label-encoded → `top_group_enc` feature.
`Customer\First\Invoice\Date`	Cross-sell	Drives the `tenure_days` feature (snapshot date − first invoice).
`Cost\of\part`	Cleaning	Used to recover `GM%` where it is null; used to filter "garbage" rows where `Cost\\TU\textgreater{}\20\×\Invoiced\price`. Not used as a feature directly.

Column	Where Used	Role in This Project
Intercompany	Cleaning	Pre-modeling filter: Intercompany\=\YES rows (intra-corporate transfers) are removed before any analysis.

In addition, the **derived** column Cluster_name (output of segmentation) feeds both downstream recommenders: as the cluster_enc feature in cross-sell, and as the personalization key for cluster-conditional item popularity in up-sell.

2.3 Columns Not Used

Every other column in the raw dataset is dropped early in cleaning or simply never read. The table below records what was excluded and why — keeping the audit transparent rather than implying every field was touched.

Column	Why Not Used
Invoice\Line\#	Position-within-invoice metadata; irrelevant once we aggregate to invoice or customer-month; almost as much unique numbers as rows
Order\#, Order\Date, Order\Line\#	Order-side artifacts of the PO cycle; Invoice\Date is the canonical timestamp for RFM and the temporal split.
Ordered\qty	Used only as a sanity check during cleaning; the canonical "the customer bought this" signal is Invoiced\qty\(shipped).
Material\cost, Labor\cost, Overhead\cost	Components of Cost\of\part; the aggregate is sufficient for our cleaning rules.
Invoiced\price\(TX)	Field semantics are unclear (likely a tax variant); excluded to avoid introducing noisy features.
Sales\Channel\-\Internal, Sales\Channel\-\External, Sales\Channel\-\Grouping	Channel attribution metadata; not a customer- or product-level feature relevant to "what should this customer buy next."
Make\vs\Buy	Supply-side attribute (made internally vs. sourced); not predictive of cross-sell behavior.
Manufacturing\Region, Manufacturing\Location\Code	Where the item is produced — supply-side, not customer-side, so irrelevant for the cross-sell and up-sell
Product\family	Superseded by Product\group, which is the chosen basket and recommendation granularity.
Born\on\date, Price\last\modified\date\in\the\ERP	Item lifecycle / pricing-update metadata; no expected signal for cross-sell or up-sell.
#\of\unique\products\on\a\quote	Quote-level metric, not aligned with the invoice-line granularity of the dataset.

2.4 Establishing the B2B Nature of the Dataset

The brief does not explicitly label the dataset, but every structural and statistical signal points to **B2B manufacturing or industrial distribution**:

Structural signals (schema): - An Intercompany flag — only meaningful when sales can be intra-corporate. - Customer\industry (e.g., IC000, IC001) — B2C buyers do not have an "industry." - Make\vs\Buy, Cost\of\part, Manufacturing\Location\Code — manufacturing vocabulary. - Top\Customer\Group, Customer\First\Invoice\Date — key-account-management terminology. - Separation of Ordered\qty and Invoiced\qty\(shipped) — classic B2B PO cycle with partial shipments. - 6-digit numeric CustomerID — typical of an SAP/Oracle/JDE customer master file.

Statistical signals (after filtering Intercompany=YES): - **11,775 unique customers** — order of thousands, not millions. - **Median customer revenue ≈ $6,737, P90 ≈ $211,738, max ≈ $1.15B.** A max/median ratio of ~171,000× is impossible in a

B2C context. - **Median 5 invoice lines per customer, P90 ≈ 87 lines** — bimodal distribution: long tail of small occasional buyers + a minority of large enterprise accounts with thousands of lines per year.

This conclusion drives several modeling decisions: customer-level (not invoice-level) market baskets, segmentation by RFM that is sensitive to whales, and cluster-conditional item rankings.

2.5 Initial Data-Quality Findings

Exploratory analysis surfaced six classes of problematic data, all of which had to be addressed before modeling:

1. **Placeholder customer ID `-99`.** Source-system NULL marker, not a real customer.
2. **`SAMPLE` and `/`-prefixed item codes.** Technical records and free samples — not real purchases.
3. **Zero / null `Invoiced\price`.** ~9% of rows. A mix of technical artifacts and recoverable values.
4. **Negative `Invoiced\price` and negative quantities.** Credit notes / returns — small in volume but distort RFM.
5. **`Cost\of\part` `Invoiced\price`.** A handful of rows where cost was 20× the price — clearly garbage.
6. **`GM%` outliers.** Tail values down to −1000+ and a few above 1.0 — needed winsorization.

Sample/slash item codes alone numbered hundreds of thousands of rows; `Intercompany=YES` removed another large block. These are quantified in §3.

---

3. Data Preparation

The cleaning pipeline runs entirely in `notebooks/data_cleaning.ipynb`. Every decision was made deliberately, with the reasoning captured below.

3.1 Encoding Fix and Initial Column Drop

The notebook's first step converts the raw `LUMEN_DS.csv` from UTF-16 → UTF-8 and strips embedded double quotes that would otherwise break the pipe parser. The result is cached as `data/raw/data.csv` so the conversion is skipped on subsequent runs. Then 11 columns are dropped because they are redundant or too granular for any of the planned models:

Dropped Column	Reason
`Invoice\Line\#`	Order within an invoice — irrelevant for recommendations
`Order\Date`, `Order\Line\#`, `Order\#`	`Invoice\Date` is the canonical time stamp
`Material\cost`, `Labor\cost`, `Overhead\cost`	Aggregated in `Cost\of\part`
`Sales\Channel\-\Grouping`	Redundant with `Sales\Channel\-\Internal/External`
`Born\on\date`, `Price\last\modified\date\in\the\ERP`	Item lifecycle metadata, not predictive of cross-sell
`#\of\unique\products\on\a\quote`	Quote-level metric, not invoice-level

3.2 Intercompany Filter

`Intercompany=YES` rows represent intra-corporate transfers, not real customer transactions. They were filtered out before any other analysis to avoid polluting RFM and basket statistics.

3.3 Row-Level Cleaning Decisions

3.3.1 Drop placeholder customer ID `-99` Source-system NULL. Removing these rows is unambiguous.

3.3.2 Drop `SAMPLE` and `/`-prefixed item codes These are free samples (not requested purchases) and source-system technical records, respectively. They appear on hundreds of thousands of lines and would create phantom co-purchase signals. Filter:

```
Item Code ~ "SAMPLE" or Item Code starts with "/"
```

3.3.3 Handle zero / null `Invoiced\price` Roughly 9% of rows had a zero or null price. Two options: drop or impute. We chose a **threshold-based imputation** strategy:

- For each `Item\Code`, compute the share of its rows that have a positive price.
- If ≥ 50% of an item's rows have a valid price (chosen via sensitivity analysis at thresholds 10/30/50/70/90%), impute the null/zero rows with the **median of positive prices for that item**.
- If < 50% of an item's rows have a valid price, the item is too unreliable to impute — drop those rows.

This balances data retention against contamination. The 50% threshold preserves the bulk of recoverable rows while excluding items where the price field is mostly broken.

3.3.4 Drop negative `Invoiced\price` Negative invoiced price rows are credit notes (returns / refunds). Including them as "purchases" would corrupt Monetary and basket signals.

3.3.5 Drop negative quantities; split out zero-shipped rows

- Negative `Invoiced\qty\(shipped)` and negative `Ordered\qty` are returns — dropped.
- Rows with `Invoiced\qty\(shipped)\==\0` were separated into a side dataframe (intent-but-not-shipped) but not used downstream because the canonical signal for "the customer bought this" is a non-zero shipped quantity.

3.3.6 Drop garbage cost rows Rows where `Cost\of\part\\TU\textgreater{}\20\×\Invoiced\price` are data-entry errors (e.g., wrong unit conversion). They distort `GM%` toward extreme negative values. Filter is conservative — anything 20× the price is clearly broken; legitimate low-margin items are not affected.

3.3.7 Winsorize `GM%` to `{[}-5,\1{]}` After fixing the cost-vs-price garbage, the residual `GM%` distribution still has a long left tail and a few values above 1.0 (price = 0 edge cases that escaped earlier filters). Clipping to `{[}-5,\1{]}` retains the signal direction while eliminating noise.

3.4 Imputation of Categorical Nulls

The following columns are filled with the literal string `\XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{}Unknown\XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{}`:

```
Manufacturing Region, Manufacturing Location Code, Intercompany,
Customer industry, Customer Region, Top Customer Group,
Product family, Product group, Make vs Buy,
Sales Channel - Internal, Sales Channel - External
```

Reasoning: `\XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{}Unknown\XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{}` becomes its own category in label encoding, preserving the "missingness" as a feature rather than silently introducing implausible values via mode-imputation.

3.5 GM% Recovery Where Possible

Where `GM%` was null but both `Invoiced\price\≠\0` and `Cost\of\part` were present, GM% was recomputed from first principles:

```
GM% = (Invoiced price - Cost of part) / Invoiced price
```

This recovers a meaningful share of rows. The remaining null `GM%` is left as-is — those rows lack one of the inputs (typically `Cost\of\part` is null across all rows for a given item, so no imputation source exists).

3.6 Final Clean Dataset

The pipeline output (`data/processed/data_clean_final.csv`) is the canonical input for **every downstream notebook**. The cleaning steps reduce the dataset from ~1.3M raw invoice lines to **~930K rows** retained for modeling. After the steps above:

- All key identifiers (`CustomerID`, `Item\Code`, `Invoice\Date`, `Invoiced\price`) are populated.
- Prices are all ≥ 0 and quantities all > 0.

- GM% is bounded in {[}-5,\1{]}.
- Categorical nulls are explicit \XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{}Unknown\XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{} values.

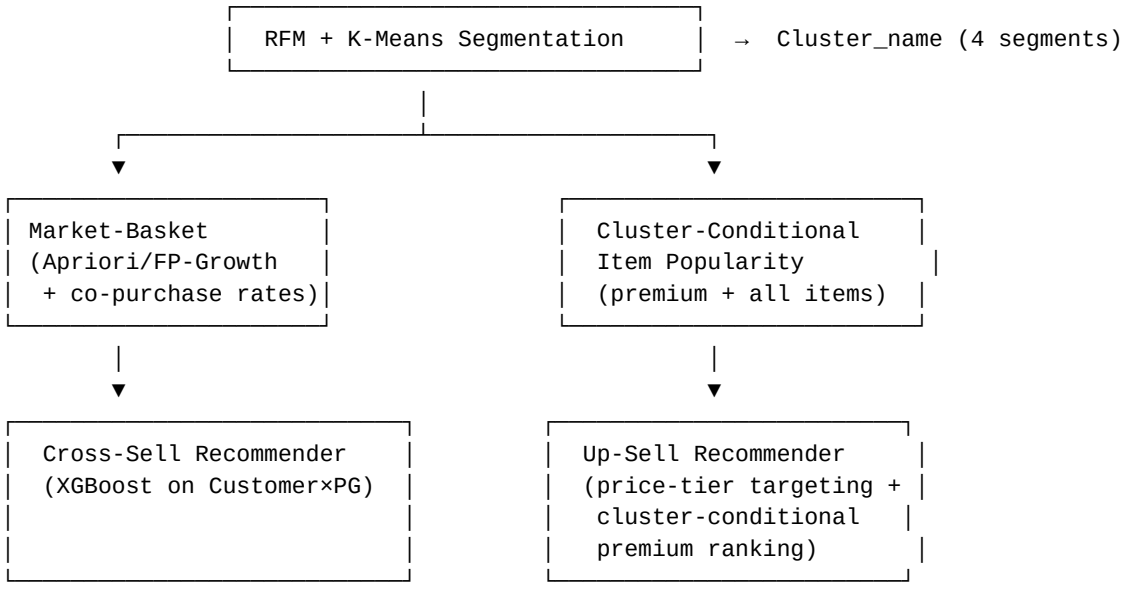
3.7 Archived Exploration — encoding_and_feature_engineering.ipynb

An earlier notebook (notebooks/archive/encoding_and_feature_engineering.ipynb) label/one-hot encoded the cleaned dataset and produced a 928k-row encoded file plus customer-level aggregate tables. It was useful as an exploration step — confirming feature distributions and aggregates — but the production pipeline does **not** depend on it: each downstream notebook re-reads data_clean_final.csv and rebuilds the features it needs inline. The archived notebook is kept for transparency but is not part of the API path.

---

4. Modeling

The recommendation system has three modeling components that build on one another:



All temporal models use a fixed split:

- **Train period:** invoices before **2019-07-01** (~2.5 years of history)
- **Label window:** invoices from **2019-07-01 to 2020-07-01** (12-month forward window)

Random seeds are fixed (random_state=42) on every model and on SMOTE.

4.1 Customer Segmentation — RFM + K-Means

Notebook: notebooks/customer_segmentation.ipynb.

4.1.1 RFM Computation Snapshot date: **2020-07-01** (one day after the maximum invoice date in the cleaned dataset).

Per customer: - **Recency** — days from the customer's most recent invoice to the snapshot date (lower is better) - **Frequency** — number of distinct invoices in the customer's history - **Monetary** — sum of Invoiced\price\×\Invoiced\qty\(shipped) across the customer's history

Output: 11,753 customers with full RFM scores.

4.1.2 RFM Scores (1–5) and Rule-Based Segments Each of R, F, M is binned into quintiles using pd.qcut (Recency reversed: lower is better → score 5). Customers are then assigned a human-readable segment based on the canonical R-F

mapping (Champions, Loyal, At Risk, Hibernating, etc.). These rule-based labels are kept as descriptive columns alongside the K-Means cluster.

4.1.3 K-Means Clustering Inputs are log-transformed (`log1p`) and standardized (`StandardScaler`) to compress the heavy right tails in Frequency and Monetary.

Choice of K: elbow + silhouette analysis over `K\␣\{[]2,\7{]}` (`n_init=10`, `random_state=42`). K=4 was selected as the best tradeoff between silhouette score and cluster interpretability.

Final clusters and business interpretation:

Cluster	Name	Profile	Business Meaning
1	**Whales / Champions**	Recent, frequent, very high spend	Enterprise key accounts — highest-revenue tier. Protect the relationship; recommend new categories cautiously.
3	**Active Small Buyers**	Recent, modest frequency, modest spend	Healthy core customers. Cross-sell candidates with the highest hit rate per outreach.
2	**Mid-Tier At Risk**	Moderate recency, moderate spend, declining engagement	Win-back targets. Personalized cross-sell can re-activate them.
0	**Hibernating**	Long recency, low frequency, low spend	At highest churn risk. Cheap to re-engage but lowest expected value per offer.

`Cluster_name` is exported in `data/processed/rfm_segments.csv` and consumed by both the cross-sell recommender (as a feature) and the up-sell model (for cluster-conditional item ranking).

4.2 Market-Basket Analysis

Notebook: `notebooks/market_basket.ipynb`.

4.2.1 Granularity Decision: Customer × Calendar Month The first instinct is to use `Invoice\#` as the basket. We tested it:

> **98.2% of invoices contain only one product group.** Multi-PG invoices are 1.8% of all invoices — too sparse to produce meaningful association rules.

Customers' purchasing rhythms in B2B are PO-driven and typically monthly (median gap between consecutive invoices ≈ 20 days for repeat customers). Re-defining the basket as **`CustomerID\+\YYYY-MM`** raises the multi-PG rate to **27.5%** — enough signal for Apriori/FP-Growth to find rules.

This is a deliberate, B2B-specific methodological choice. In a B2C retail context, invoice-level baskets would be the right unit.

4.2.2 Granularity Decision: Product Group, Not Item Code Item Code has ~52,000 unique values; rules at that granularity would have near-zero support and overfit on noise. Product Group (~30 values) is the natural recommendation level — it is what the sales team thinks about and what an e-commerce site categorizes by. A bonus Item-Code-level analysis in the notebook confirmed that PG is the right granularity.

4.2.3 Algorithms: Apriori and FP-Growth Both algorithms were run with `min_support=0.01`, `max_len=3`. As expected, they produced **identical frequent itemsets** (verified in-notebook). FP-Growth is preferred for performance; Apriori is kept for didactic clarity (the antimonotonicity property that justifies pruning).

4.2.4 Rule Filtering Rules were filtered by three thresholds:

Metric	Threshold	Reasoning
Lift	> 2.0	Rules at least 2× stronger than chance — clearly non-random associations
Confidence	> 0.30	At least 30% probability of buying the consequent given the antecedent
Conviction	> 1.3	Disagreement with the independence assumption — robustness check

Result: 14 strong association rules (`data/processed/association_rules.csv`).

4.2.5 Co-Purchase Rates as a Feature Extension The 14 hard rules cover only ~0.6% of all directed PG-to-PG pairs. To extend the signal, we also computed `P(PG_b\\TU\textbar{}\PG_a)` for **all** observed directed pairs (up to 870 = 30 × 29). This produces 27 features per (customer, candidate-PG) pair in the cross-sell recommender:

- `max_copurchase_rate` — strongest co-purchase signal from any of the customer's existing PGs
- `mean_copurchase_rate` — average co-purchase signal across all existing PGs

This means the cross-sell model has both **strong-rule** features (from the 14 rules) and **soft-signal** features (from the 870 co-purchase pairs).

4.3 Cross-Sell Recommender — Customer × Product Group Model

Notebook: `notebooks/cross_sell_recommender.ipynb`.

4.3.1 Problem Framing Earlier work (`notebooks/archive/cross_sell_model.ipynb`) framed cross-sell as a customer-level binary classification: "will this customer ever cross-sell?" That answers a question, but not a useful one — it doesn't tell the API *which* product to recommend.

The production recommender reframes the task as a **per-pair prediction**:

> For each (customer, candidate product group not yet bought in the train period), predict the probability that the customer will buy from that group in the 12-month label window.

This produces a **ranked list per customer** that maps directly to the API output.

4.3.2 Matrix Construction For each customer in the train period: - Get the set of PGs they already bought in the train period (these are excluded — we only want to recommend new categories). - For every other PG in the catalog, create a row. - Target = 1 if the customer bought from that PG in the label window, 0 otherwise.

Result: ~257K (customer, PG) pairs, **0.34% positive class rate** (≈ 290:1 imbalance). This imbalance is a fundamental property of the problem: most cross-sell opportunities don't convert in any given year.

4.3.3 Feature Engineering — 27 Features per Pair Three families of features:

Customer features (15) — all computed from train period only: - Volumetric: `train_revenue`, `train_orders`, `train_items`, `train_n_groups` - Behavioral: `train_avg_gm`, `recency_days`, `tenure_days`, `order_frequency` - Categorical (label-encoded): `region_enc`, `industry_enc`, `top_group_enc`, `cluster_enc` - RFM scores: `R_score`, `F_score`, `M_score`

Product-group features (7): - `pg_n_customers`, `pg_avg_price`, `pg_avg_gm`, `pg_total_orders`, `pg_avg_revenue`, `pg_popularity`, `pg_enc`

Association/co-purchase features (5): - From the 14 hard rules: `rule_exists`, `rule_max_lift`, `rule_max_confidence` - From the 870 co-purchase pairs: `max_copurchase_rate`, `mean_copurchase_rate`

4.3.4 Imbalance Handling — SMOTE A 290:1 imbalance is too extreme for `class_weight` alone. **SMOTE** with `sampling_strategy=0.1` is applied **only on the training fold** (using `imblearn.pipeline.Pipeline` so the test fold remains untouched). This raises the train-fold positive rate from 0.34% to ~9%, enough for tree-based models to find a clean decision boundary.

4.3.5 Model Comparison Three models trained on the same feature set with identical SMOTE:

Model	Test ROC-AUC	5-Fold CV ROC-AUC	Notes
Logistic Regression	0.8654	—	Baseline — linear in features
Random Forest	0.9141	—	Captures non-linear interactions
XGBoost	**0.9248**	**0.9291 ± 0.0055**	Best — tightly bounded variance across folds

5-fold cross-validation uses `GroupKFold` keyed on `CustomerID` so all pairs for a given customer fall into the same fold — preventing customer-level leakage between train and validation.

The XGBoost configuration:

```
n_estimators=300, max_depth=6, learning_rate=0.05,
subsample=0.8, colsample_bytree=0.8, random_state=42
```

XGBoost is selected as the production model. Its `.pkl` is saved to `models/recommender/recommender_xgb.pkl` and is loaded by the API.

A subsequent rolling-window experiment in §13 of the notebook retrained XGBoost on three additional historical windows (Jul–Dec 2018, Oct 2018–Mar 2019, Jan–Jun 2019) stacked onto the base train period. The expanded training did **not** beat the base model — AUC stayed at the same level — so the deployed `recommender_xgb.pkl` is the pre-rolling-window, 27-feature XGBoost reported above (AUC 0.9248). The rolling-window code is kept in the notebook for reproducibility but is not part of the production artifact.

4.3.6 Layer 2 — Item Suggestions Within a Recommended PG The XGBoost model ranks **product groups**, but the API also returns **specific item codes** within each recommended group. This is the "Layer 2" lookup — it reuses the up-sell notebook's cluster-conditional item popularity (without the premium-only filter) to recommend the most popular items in the recommended PG **among customers in the same cluster**.

This means a Whale and a Hibernating customer get the same set of recommended PGs but potentially different item suggestions within each PG.

4.4 Up-Sell Model

Notebook: `notebooks/upsell_model.ipynb`.

4.4.1 Tier Classification — Standard vs Premium For each `(Product\group, \Item\Code)`, compute the item's average price across the train period. Then within each PG, take the **median of those item averages**. Items at or above the median are tagged **`premium`**; below are **`standard`**.

Result: 38,734 items tiered, of which 19,382 are premium (the top half of every PG by item-average price).

The PG median of item averages — not of raw line prices — was chosen so that volume-skewed cheap items don't drag the threshold down.

4.4.2 Up-Sell Target Identification For each (customer, PG) pair, compute the customer's average price paid in that PG during train. If their average is **below the PG median**, they are an up-sell target — they are buying in the category but at the lower-tier prices.

Result: 6,458 (customer, PG) target pairs across 4,993 unique customers.

This is the up-sell candidate pool. Note that the API also handles the "current item is already premium" case by returning no recommendation.

4.4.3 Cluster-Conditional Premium Item Popularity For each `(Cluster_name,\Product\group,\Item\Code)` combination where the item is `premium`:

```
confidence = (# customers in cluster who bought this item in this PG)
           / (# customers in cluster who bought anything in this PG)
```

A `n_buyers\≥\2` filter removes single-purchase noise. (One known edge case: PG `SF001` has only one item, so its rankings degenerate to "buy the only item available" — flagged in the notebook.)

This gives a per-cluster "what premium items do my peers buy?" ranking, which is the personalization layer for up-sell recommendations.

4.4.4 Recommendation Generation For each up-sell target (customer × PG): 1. Look up the customer's cluster. 2. From that cluster's premium item popularity in the target PG, drop items the customer has already bought. 3. Take the top-K (K=5) by confidence.

Result: 27,744 up-sell recommendations covering 4,843 customers across 26 product groups.

---

5. Evaluation

5.1 Segmentation Evaluation

- **Cluster sizes** are well-balanced (no single cluster swallows the population, no cluster is degenerate). All four clusters are populated by hundreds-to-thousands of customers.
- **Snake plot** (per-cluster mean of scaled R, F, M) shows clearly different profiles — Whales sit high on F and M with low R; Hibernating is the mirror image; the two mid-tier clusters are crisply separated by recency.
- **Interactive 3D plot** (`visualization/clusters_3d.html`) confirms visually that the four clusters occupy distinct regions of the log-scaled RFM space.
- **Silhouette score** at K=4 was higher than at K=3 and competitive with K=5/6 while remaining far more interpretable to the business.

The clusters are not just statistically separable — they map onto recognizable business archetypes (key accounts, healthy core, at-risk, dormant), which is the harder bar to clear.

5.2 Cross-Sell Recommender Evaluation

5.2.1 Primary Metric: ROC-AUC

Setting	Score
Test set ROC-AUC (XGBoost)	**0.9248**
5-fold GroupKFold CV ROC-AUC	**0.9291 ± 0.0055**
Random Forest test AUC	0.9141
Logistic Regression test AUC	0.8654

The cross-validation interval is tight (±0.005), which means the AUC is stable across different customer subsets — not a fluke from one favorable train/test split.

5.2.2 Secondary Metric: Precision@5 For each test-set customer, take the model's top-5 recommended PGs and compute the share that the customer actually bought in the label window.

- **Average Precision@5: 0.38%**
- **Random baseline (290:1 imbalance): 0.34%**

The improvement over random is small in absolute terms — this is a property of the underlying class imbalance and the long-tail nature of B2B cross-sell, *not* a model failure. AUC is the appropriate metric for ranking systems with extreme imbalance; Precision@5 is reported as a sanity check, not as the headline figure.

5.2.3 Feature Importance (Random Forest, qualitative) The strongest features cluster into three groups, in roughly this order of importance:

1. **Co-purchase signal** (`max_copurchase_rate`, `mean_copurchase_rate`) — extending the 14 hard rules to the full 870-pair landscape was high-leverage.
2. **Customer behavioral features** (`train_revenue`, `train_orders`, `train_n_groups`, `recency_days`).
3. **Product-group base rates** (`pg_popularity`, `pg_n_customers`).

The hard-rule features (`rule_exists`, `rule_max_lift`, `rule_max_confidence`) are useful but cover too few pairs to dominate.

5.2.4 Validation Methodology — Why Temporal Hold-Out Random splits on (customer, PG) pairs would let the model see one PG for a customer in training and predict another for the same customer in test — that's not a real-world cross-sell scenario. **Temporal hold-out** (train < 2019-07-01, label 2019-07-01 → 2020-07-01) is the only way to honestly answer "given what I know about a customer today, what will they buy in the next 12 months?" `GroupKFold` on `CustomerID` is the cross-validation analog of the same principle.

5.3 Up-Sell Model Evaluation

The validation is harder because there is no direct "did this customer accept the up-sell?" signal. We use a **forward-time price-uplift proxy**:

> If a customer was a target in the train period (buying below the PG median) and reappears in the label window, did their average price in that PG go up?

Headline numbers (from `models/upsell/upsell_meta.json`):

Metric	Value	Interpretation
Targets with label-window data	3,720 / 6,458 (59.9%)	Coverage of the proxy metric
Average price uplift (label vs. train)	**+$35.62**	Targets paid more on average in the next 12 months
Share with positive uplift	**44.25%**	Nearly half moved up in price within the same PG
Share that crossed into the premium tier	**7.61%**	Hard signal — they actually bought a premium item

These numbers are presented honestly with their limitations:

- **No control group.** We cannot separate model effect from natural price drift in the market. The notebook flags this and proposes computing the same migration rate on a random sample of non-targeted customers as a sanity check.
- **Coverage is 60%, not 100%.** The 40% of targets who do not reappear in the label window contribute neither evidence nor counter-evidence.

Read together: **the up-sell targeting clearly identifies a segment whose price behavior moves upward in the forward window.** Whether that movement is causal or correlational requires an A/B test; the proxy is a strong-enough signal to justify deploying the recommender.

5.4 API Recommendation Quality

Beyond model metrics, the API's recommendation quality is observable in two ways:

1. **Coverage.** With 9,017 known customers loaded into memory and 30 product groups, every existing customer always receives 5 recommendations. Cold-start (unknown CustomerID or no `customer_id` passed) falls back to population medians and still produces ranked output.
2. **Personalization.** Two customers in different clusters who receive the same recommended PG see different `items` lists — driven by `upsell_item_popularity_all.csv` filtered by `Cluster_name`.

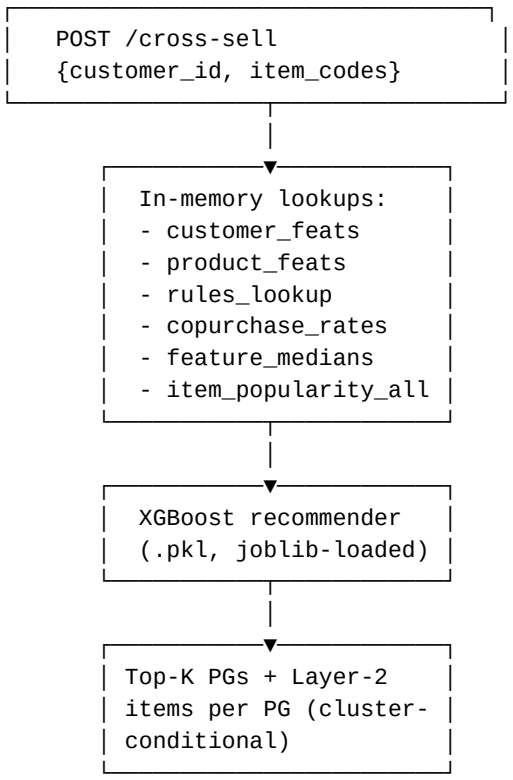
Live latency (after warm startup) is sub-second per call because every artifact is loaded into a memory dictionary at startup; no disk reads happen during request handling.

---

6. Deployment

6.1 Architecture

The deployment is a **single-process FastAPI application** (`api/main.py`) that loads all model artifacts and lookup tables into memory at startup via the `lifespan` hook.



6.2 Endpoints

`GET\/health` Sanity check that the model is loaded and customer features are in memory:

```
{ "status": "ok", "model_loaded": true, "known_customers": 9017, "layer2_available": true }
```

`POST\/cross-sell` **Input** | Field | Type | Required | Behavior if Missing | |—|—|—|—| | `customer_id` | int | No | Falls back to population-median features | | `item_codes` | list[str] | No | No co-purchase / association signal added |

Logic 1. Load the customer's stored features (or median fallback) and `Cluster_name`. 2. Map provided `item_codes` to their PGs via the `item_tiers` lookup. 3. Build candidate set: all PGs **minus** ones the customer already buys **minus** PGs from the current order. 4. For each candidate PG, build the 27-feature row at request time (rule features and co-purchase features depend on the live `item_codes`, so they are not pre-computed). 5. Score with XGBoost, sort by probability, take top 5. 6. For each top-K PG, look up Layer-2 items in the customer's cluster.

Output

```
{
  "customer_id": 12345,
  "recommendations": [
    {
      "product_group": "PC016",
      "confidence": 0.4642,
      "items": [
```

```
        {"item_code": "BOE001-TY8.375", "confidence": 0.1098, "avg_price": 6.02}
      ]
    }
  ]
}
```

`confidence` on the PG is the XGBoost-predicted probability the customer will buy from that group. `confidence` on each item is the cluster-conditional buyer share for that item within the PG.

`POST\/up-sell` **Input** | Field | Type | Required | |—|—|—| | `customer_id` | int | No (skips cluster personalization if missing) | | `item_codes` | list[str] | Yes |

Logic For each input item code: 1. Look up the item's tier and PG (skip if unknown). 2. If the item is already `premium`, skip — there is nothing to up-sell. 3. Filter `upsell_popularity` to the same PG, then optionally to the customer's cluster (if a cluster is found; otherwise serve cluster-agnostic). 4. Take top-K premium alternatives by confidence, excluding the item the customer is already buying.

Output

```
{
  "customer_id": 12345,
  "recommendations": [
    {
      "current_item": "03650",
      "product_group": "PC001",
      "current_tier": "standard",
      "current_avg_price": 0.0,
      "premium_alternatives": [
        {"item_code": "2158", "confidence": 0.0495, "avg_price": 78.99}
      ]
    }
  ]
}
```

6.3 Cold-Start Behavior

Both endpoints accept missing `customer_id` (B2B walk-up case or first-time customer):

- **Cross-sell:** features fall back to `feature_medians.json` (population medians from training); cluster-specific Layer 2 items fall back to cluster-agnostic ranking. Result: a defensible "what would the average customer want next?" answer.
- **Up-sell:** cluster personalization is skipped, and the cluster-agnostic premium-item ranking within the PG is returned.

6.4 Running in Production

Reference command (development):

```
conda activate lumen-ds
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
```

Production deployment notes:

- **Single-worker assumption.** `state:\dict` is in-process; multi-worker (Gunicorn) deployment would need either a shared cache or per-worker reload of the same artifacts.
- **No authentication or rate limiting.** Both endpoints are open by design (the API is internal). Add authentication at the gateway layer if the API is exposed beyond the company network.
- **No model versioning beyond `model_meta.json`.** When models are retrained, the meta JSON is overwritten — there is no automatic record of which dataset hash produced which `.pkl`.
- **API requirements are version-pinned** (`api/requirements.txt`) — the notebook environment is documented in the README/Run Book.

6.5 Reproducibility

Every step in the pipeline uses fixed seeds and deterministic split dates:

Knob	Value	Where Set
Random seed	42	All sklearn/XGBoost/SMOTE constructors
Train cutoff	2019-07-01	Cross-sell, up-sell notebooks
Label end	2020-07-01	Cross-sell, up-sell notebooks
RFM snapshot	2020-07-01	Customer segmentation
K (clusters)	4	Customer segmentation
SMOTE strategy	0.1	Cross-sell recommender
Top-K	5	Both endpoints, both training notebooks

A minor source of variance — XGBoost `n_jobs=-1` can produce ±0.002 AUC variation due to non-deterministic floating-point accumulation in parallel — is documented in the Run Book and does not affect business decisions or rankings.

Appendix — Pipeline Output Summary

Notebook	Primary Output	Used By
`01\data_cleaning`	`data/processed/data_clean_final.csv`	Every downstream notebook
`02\customer_segmentation`	`data/processed/rfm_segments.csv`	Cross-sell recommender, up-sell model
`03\market_basket`	`data/processed/association_rules.csv`	Cross-sell recommender
`04\cross_sell_recommender`	`models/recommender/*.pkl`, `data/processed/recommender/*.csv`, `feature_medians.json`	API `/cross-sell`
`05\upsell_model`	`data/processed/upsell/*.csv`, `models/upsell/upsell_meta.json`	API `/up-sell` and `/cross-sell` Layer 2
`archive/encoding_and_feature_engineering`	`customer_features.csv`, `data_clean_enc_fe.csv`, `invoice_items.csv`	Exploratory only — not consumed by the production pipeline (see §3.7)
`archive/cross_sell_model`	`models/cross_sell/*.pkl`	Exploratory only — superseded by the production cross-sell recommender (see §4.3.1)
